Identifying the Flight Phase of Cellular-Connected UAVs from Encrypted Command-and-Control Traffic: A Federated Learning Approach Based on Differential Attention

Abstract The use of cellular networks for the command-and-control (C2) link of unmanned aerial vehicles (UAVs) has grown rapidly over the past few years, and the C2 traffic is commonly protected by an encrypted tunnel such as WireGuard or IPsec. It is generally assumed that, once the payload is encrypted, a third party observing the network cannot learn what the aircraft is doing. In this paper, we show that this assumption does not hold. We consider a passive observer located at a cell site who sees only the size, direction and arrival time of the encrypted packets, and we study whether such an observer can identify the flight phase of the UAV, i.e., whether the aircraft is on the ground, flying an autonomous mission, or being flown manually by an operator. To obtain labelled traffic, we built an emulation testbed in GNS3 in which the unmodified ArduPilot firmware flies randomised missions for two fleets of 12 multirotor and 12 vertical take-off and landing aircraft, over WireGuard and IPsec respectively, through emulated 5G, LTE and satellite cell sites with realistic rate limits, delay, jitter, loss and competing user traffic. Each cell router records the encrypted traffic while each aircraft logs its true state. Since the captures are distributed over several observation sites that cannot share their data, we propose DAPPER, a federated learning framework in which a gated differential-attention encoder is trained over statistical and temporal features of the traffic; only the encoder is averaged between sites, while a small set of site-specific calibration parameters and prompts (780 parameters per site) is kept local. Experimental results on held-out flights show that the proposed method identifies the operational state with a macro-F1 of 0.998 and 0.984 on the two fleets, which is within 0.5 points of a centralised model trained on all the data, and that it can be applied to a new cell site without any labelled traffic from that site. The accuracy decreases for finer-grained phases (0.838 and 0.757 for four operational phases, 0.486 and 0.506 for seven kinematic phases), because the telemetry is transmitted at a fixed rate and only sparse mission events differ between these phases. The main result is further confirmed on the public PiNcH dataset of a real drone over encrypted Wi-Fi. Finally, we evaluate three traffic-shaping countermeasures and find that packet padding and random delays do not prevent the attack, whereas constant-rate cover traffic does, at a cost of about 250% additional bandwidth. The testbed, the dataset and the source code are publicly released.

Keywords: encrypted traffic analysis, unmanned aerial vehicles, flight-phase inference, federated learning, differential attention, side-channel privacy, MAVLink, network emulation

Introduction

Unmanned aerial vehicles (UAVs) are increasingly used for delivery, inspection, mapping and public-safety applications, and many of these applications require the aircraft to fly beyond the visual line of sight of its operator. In such deployments the command-and-control (C2) link is no longer a short-range radio link to a handheld controller; instead, the aircraft is connected to a cellular network, and a ground-control station (GCS) located anywhere on the Internet supervises the flight [1, 2]. The most widely used control protocol for this link, MAVLink, was designed for a trusted serial channel and does not provide confidentiality by itself. Operators therefore tunnel the MAVLink traffic inside a standard encrypted transport, such as WireGuard or IPsec, or protect it with a dedicated cipher [3, 4]. It is commonly assumed that, once the C2 payload is encrypted, a third party who observes the traffic, for instance at a base station or in a transit network, cannot learn anything useful about the aircraft.

In this paper, we show that this assumption is not valid. Encryption protects the content of the MAVLink messages, but it does not hide their size, their direction and their timing. The autopilot streams position, attitude and status messages at rates requested by the GCS; the operator transmits set-points or stick commands only while flying manually; a mission is uploaded before an autonomous flight; and a companion computer uploads a log file whose size depends on the flight duration. Each of these events leaves a characteristic pattern in the encrypted traffic. We study to what extent a passive network observer can exploit these patterns to identify the flight phase of the aircraft, i.e., whether it is on the ground, flying an autonomous mission or being flown manually by an operator, and, at a finer level, whether it is hovering, cruising, orbiting, climbing or descending.

Flight-phase identification is different from the two problems that have been addressed in the literature on UAV traffic analysis, namely detecting the presence of a drone and identifying its model. Knowing the flight phase reveals the intention of the operator and exposes the aircraft at its most vulnerable moments. For example, an adversary who detects that the aircraft has entered a stationary hover or a low-altitude orbit knows that it is loitering over a target and can be located, jammed or captured more easily; an adversary who detects the beginning of manual control knows that an operator is in the loop; and an adversary who detects the descent phase knows that a landing site is about to be used. None of these inferences requires breaking the encryption.

Three characteristics of real deployments make this problem more difficult than a laboratory capture would suggest. First, the C2 traffic is mixed with the traffic of other users and is affected by the link conditions. A cell site serves many users, the C2 flow shares a rate-limited backhaul with unrelated uplink and downlink sessions, and the delay, jitter and loss of the link vary considerably between a 5G small cell, a congested LTE macrocell and a geostationary satellite link. Second, the traffic pattern of the same flight phase differs from one observation site to another, because the retransmissions of the autopilot, the stream-rate negotiation of the GCS and the keep-alive messages of the tunnel all depend on the path; a classifier trained at one site performs worse at another. Third, the captures are distributed over several observation sites and cannot be collected in one place, since the observers, whether a fleet operator auditing its own exposure in several regions or several parties each monitoring one site, are separated by privacy, legal or commercial constraints. This is exactly the setting addressed by

federated learning [5, 6].

To address these three issues, we propose DAPPER (Differential-Attention, Path-Personalised federatEd Recognition), a federated learning framework for flight-phase identification from encrypted C2 traffic. The classifier processes each traffic window through two sets of features, namely statistics of the packet sizes and inter-arrival times over the window and a sequence of 100 ms bins, using gated differential attention [7, 8], a recent variant of the Transformer that computes the attention weights as the difference of two softmax maps and thereby reduces the weight assigned to irrelevant time bins by cross-traffic and jitter. To cope with the differences between observation sites, a small number of parameters is kept specific to each site: an input calibration that normalises the traffic statistics of the site and a set of learned prompt vectors that are prepended to the input sequence. These site-specific parameters are trained locally and never leave the site, while the shared encoder is trained with federated averaging, so that no raw traffic is exchanged.

Since no public dataset contains phase-labelled UAV traffic over a tunnelled cellular link, we built an emulation testbed in GNS3 to collect such traffic. The unmodified ArduPilot firmware runs as software-in-the-loop in one container per aircraft and tunnels MAVLink 2 over a real WireGuard or IPsec tunnel to a fleet GCS through emulated cell sites, whose links are configured with realistic rate limits, delay, jitter, loss and competing user traffic. The GCS flies each aircraft through randomised missions for several hours; every cell router captures the encrypted traffic with `tcpdump`, and every aircraft logs its true state at 5 Hz, which provides flight-phase labels that are independent of the traffic. Two fleets were collected in this way, multirotor aircraft over WireGuard and vertical take-off and landing (VTOL) aircraft over IPsec. In addition, we validate the main result on the publicly available PiNcH dataset, which contains captures from a real drone over encrypted Wi-Fi.

The main contributions of this paper are summarised as follows.

1. We show that the operational state of a cellular-connected UAV (on the ground, in autonomous flight or under manual control) can be identified from its encrypted C2 traffic by a passive observer with a macro-F1 above 0.98, for two types of aircraft, two encrypted transports and link conditions ranging from 5G to satellite, and we quantify how the accuracy decreases for finer-grained flight phases.

2. We propose DAPPER, a federated learning framework that combines gated differential attention over statistical and temporal features with site-specific calibration and prompts. On held-out flights, it achieves results within 0.5 points of a centralised model without exchanging any traffic, outperforms the shared-model federated baselines on the heterogeneous satellite-backed fleet, and can be applied to cell sites that were not seen during training. An ablation study identifies the contribution of each component.

3. We present a reproducible emulation testbed in which real ArduPilot firmware flies randomised missions over real encrypted tunnels and emulated cellular links with per-aircraft ground truth. The testbed, the dataset and the source code are publicly released, and the main result is confirmed on the public PiNcH dataset.

4. We evaluate three traffic-shaping countermeasures, namely packet padding, timing randomisation and constant-rate cover traffic, against an adversary retrained on the protected traffic, and show that only cover traffic prevents the attack, at a bandwidth cost that we report.

The remainder of this paper is organised as follows. The next section reviews the related work on UAV traffic analysis, encrypted-traffic classification and federated

learning. The proposed method section describes the threat model, the emulation testbed, the datasets and the DAPPER framework. The results section reports the experimental results on the operational state, on the finer-grained phases, on unseen sites, on the public dataset and on the countermeasures, and compares them with prior studies. The discussion section interprets the findings and their limitations, and the last section concludes the paper.

Related work

The question studied here—recovering the flight state of a drone from *encrypted* C2 traffic by a *passive* observer, across heterogeneous and distributed observation points—lies at the intersection of four lines of work.

Traffic analysis of drones

Sciancalepore et al. [9] showed that the status of a commercial drone (flying or at rest) can be inferred from the sizes and inter-arrival times of its eavesdropped, encrypted Wi-Fi controller link, and their PiNcH system [10] applied network traffic analysis to drone detection. Alipour-Fanid et al. [11] detected UAVs and identified their operation modes from encrypted Wi-Fi traffic with packet-size and timing features, and Liang and Dong [12] identified consumer drone models from encrypted traffic. Detection of spying drones from the timing of their video stream [13] and SVM-based intrusion detection on encrypted UAV Wi-Fi traffic [14] follow the same principle. These studies observe the direct Wi-Fi link between drone and controller, resolve two or three states, and evaluate at a single capture point. The present work differs in three respects: the C2 path is a tunnelled cellular connection with its own backhaul, cross-traffic and delay; the aircraft state is resolved at three granularities up to seven kinematic phases; and the models are trained and evaluated across sites with very different link conditions. Non-network side channels have also been used, for example acoustic emissions for UAV attack detection and root-cause analysis [15].

MAVLink and cellular C2 security

MAVLink carries no confidentiality of its own, and its threats and countermeasures have been surveyed [16]. Proposed remedies encrypt the protocol [17], introduce dedicated ciphers [3], formally verify hardened variants for eVTOL systems [4], or deploy in-situ honeypots [18]; attacks on the protocol itself, such as timestamp-manipulation GPS spoofing over MAVLink 2 [19], continue to be reported. For cellular-connected aircraft, the 3GPP procedures and their security threats have been analysed [20]. All of these concern the content or the integrity of the C2 messages. The leakage studied here lies in the sizes and timing that encryption preserves and is therefore not removed by any choice of cipher. Veksler et al. [21] showed that a compromised drone can leak information covertly through MAVLink; here, no cooperation of the aircraft is assumed.

Learning on UAV and encrypted traffic

Machine learning on UAV network traffic has mostly targeted intrusion detection [1, 22, 23] and GPS-spoofing detection for cellular-connected UAVs [2], which are defender-side tasks with access to the aircraft's own vantage point. Related defender-side work on vehicular and host security includes federated, explainable intrusion detection on the CAN bus of connected vehicles [24], real-time attack classification [25], and malware detection and attribution under concept drift [26, 27].

The present task is the observer-side counterpart and is a form of encrypted-traffic classification [28], where deep models over packet-size and timing sequences were developed for website fingerprinting [29, 30] and have recently adopted attention over multi-channel traffic representations [31]. Differential attention [7] has since been applied to noisy signals in other domains, including remote-sensing segmentation [32] and hyperspectral classification [33], but not, to our knowledge, to encrypted traffic.

Federated learning across heterogeneous observation points

Captures from different observation points cannot always be pooled, which is the setting of federated learning [5, 34]. Statistical heterogeneity across clients degrades FedAvg and has motivated personalised federated learning [35, 36]; FedBN [37] keeps normalisation statistics local so that each client adapts to its own feature distribution, and federated training across heterogeneous data sources has been applied to cross-corpus speech emotion recognition [38] and medical image classification [39]. Federated learning has been applied to network-traffic classification [6, 40, 41], including personalised schemes for encrypted traffic based on hypernetworks [42], multi-task learning [43] and mixtures of experts under drift [44]. DAPPER follows the FedBN principle but personalises an explicit input calibration and a set of path prompts, and applies it to UAV flight-state inference.

Traffic-shaping defences and emulation

Padding and shaping defences against traffic analysis have been evaluated for encrypted DNS and Wi-Fi device fingerprinting [45, 46]; their cost–benefit for drone C2 has not been measured. Network-level protection of UAV complexes has also been studied through moving-target defence [47]. Public UAV-traffic datasets are short Wi-Fi captures without flight-phase labels; the testbed used here runs the ArduPilot software-in-the-loop flight stack over WireGuard and IPsec tunnels and emulated cellular links in GNS3, with per-drone ground truth.

Proposed method

This section defines the adversary and the quantity it recovers, describes the GNS3 testbed that generates the labelled encrypted traffic and the public captures used for validation, and presents DAPPER: the site-local calibration, the tokenisation of the traffic, the gated differential-attention encoder, the path-personalised federated training, and the experimental protocol.

Threat model

The adversary considered in this paper is a passive, honest-but-curious observer on the path of a cellular-connected drone's C2 link, for example at the cell site, on the backhaul or at a transit network. For each packet of the target flow the observer records only its timestamp, its size and its direction (uplink from the drone or downlink from the ground-control station). It does not decrypt any payload, does not inject or modify traffic, and has no access to the aircraft, to the ground-control station or to the tunnel keys. The target flow is isolated by its outer IP addresses and ports or by its security-association identifiers, as any on-path observer can do, and individual MAVLink messages inside the tunnel are never distinguished.

The quantity the observer attempts to recover is the state of the aircraft as a function of time. Three granularities are studied, all defined from the aircraft's own

state log and independently of the traffic. The fine taxonomy follows the autopilot's kinematics: for a multirotor, ground (armed or disarmed, not airborne), ascent, hover, cruise (horizontal translation), orbit (circling a point of interest), tele-operation (manual stick or velocity control) and descent; for a fixed-wing VTOL aircraft, ground, VTOL climb, VTOL descent, cruise (wing-borne forward flight), loiter (circling) and tele-operation. The operational phases group these into four states common to both airframes: on the ground, stationary over a point (hover, orbit or loiter), in transit (climbing, cruising or descending) and under tele-operation. The operational state, which is the primary target, merges the two airborne autonomous phases, leaving three states: on the ground, in autonomous flight and under tele-operation. The three taxonomies are mutually exclusive at any instant.

The observer may train a model offline on labelled traffic from drones and sites under its control and then apply it to a target flow. Training data from different sites cannot be centralised, so the observer, or a coalition of observers, trains federatively. Evaluation uses held-out flights: the test traffic comes from sorties, and in the unseen-site experiment from sites, that were never seen during training. Formally, let the target flow be the sequence of its outer packets, each a triple (timestamp, size, direction). A window of T consecutive 100 ms bins covers the packets of 12.8 s and carries a label $y \in \{1, \dots, C\}$, the state the aircraft occupies for the majority of the window at the chosen granularity. The observer seeks a classifier f_θ from the packets of a window to Δ^C that maximises recovery on windows from unseen flights, where θ is trained federatively across sites that never exchange raw windows. Leakage is quantified by macro-averaged F1, so that rare states count equally, by the one-versus-rest ROC area of each state, and by the confusion structure among states.

Emulation testbed

Figure 1. The GNS3 testbed and the leakage it exposes. Top: two operator networks—12 ArduCopter multirotors whose MAVLink 2 C2 runs over WireGuard (SkyNet-A) and 12 ArduPlane VTOL aircraft whose C2 runs over IPsec ESP (SkyNet-B)—each spread over four cell sites with different link conditions; every cell router passively captures the encrypted traffic of its drones. Bottom: mean uplink packet-size signature per flight phase (packets per 100 ms bin against packet size); the phases separate in the small-packet region that carries the MAVLink control and telemetry cadence.

The testbed runs inside GNS3 with Docker nodes on a Linux hypervisor. Two operator networks are emulated side by side. **SkyNet-A** is a fleet of 12 multirotors running the ArduCopter firmware; its C2 link is MAVLink 2 carried inside a WireGuard tunnel. **SkyNet-B** is a fleet of 12 fixed-wing vertical-take-off-and-landing (VTOL) aircraft running ArduPlane; its C2 link is MAVLink 2 carried inside an IPsec ESP tunnel with AES-GCM. Each fleet is divided over four cell sites with distinct link characteristics. For SkyNet-A the sites model an urban 5G small cell, a suburban LTE cell, a rural LTE cell, and a congested LTE macrocell; for SkyNet-B they model a 5G standalone cell, an LTE cell, a low-earth-orbit (LEO) satellite backhaul with a time-varying propagation delay and periodic handover outages, and a geostationary (GEO) satellite backhaul with a ∼270 ms one-way delay. Each site is a router that shapes its link with the Linux `netem` and token-bucket queueing disciplines to impose the appropriate rate limit, mean delay, delay jitter, and loss, and that carries competing cross-traffic from an emulated population of other user equipment generating on/off uplink and downlink sessions that share the site's rate-limited backhaul.

The drone nodes do not replay a trace: each runs the unmodified ArduPilot software-in-the-loop (SITL) binary—the identical flight-control code that runs on real vehicles—with its full physics model, estimators, and parameter set. A companion process on each drone forwards every MAVLink frame the autopilot produces through the encrypted tunnel to the fleet ground-control station as a companion computer does and runs the ancillary applications a fielded aircraft carries: a network Remote-ID position broadcast while airborne, periodic fleet heartbeats, time synchronisation, a post-landing flight-log upload whose size grows with the sortie duration, and occasional software-update checks. The ground-control station runs an autonomous mission supervisor that, for each drone independently and forever, draws a random operator-specific mission and flies it: for the multirotors, deliveries, lawn-mower surveys, orbital inspections, and manual tele-operation; for the VTOL aircraft, surveys, multi-leg patrols, orbital inspections, and manual tele-operation. Mission parameters—altitudes, speeds, distances, leg counts, orbit radii, and tele-operation durations—are randomised per sortie, so that no two flights produce identical traffic. Figure 2 shows the running testbed as it appears in the GNS3 web interface: the project map, the autopilots in flight, the WireGuard and IPsec tunnels, the link shaping of a satellite site, the encrypted packets seen by the cell routers, and the mission supervisor.

Figure 2. The testbed in operation (GNS3 web interface). (a) Project map with all 44 nodes running. (b,c) Consoles of a multirotor and of a VTOL aircraft behind the LEO site: the ArduPilot SITL state reported every 5 s during flight. (d) WireGuard tunnel of a multirotor. (e) IPsec ESP security associations of a VTOL aircraft (key material hidden). (f) Link shaping of the LEO satellite site. (g,h) Live packet traces at two cell routers: only encrypted WireGuard (UDP 51820) or ESP packets are visible to the observer. (i) Mission events of ground-control station A.

Capture and ground truth

Every cell router captures the traffic of each drone attached to it with `tcpdump`, recording only packet headers (the payloads are encrypted and are never stored). Because the observer sees only the outer, encrypted packets, the capture contains exactly the information the threat model allows: timestamp, size, and direction. Independently, each drone logs its own true kinematic state—armed flag, flight mode, altitude, velocity components, the active mission command, and whether a manual control stream is present—at 5 Hz, read directly from the MAVLink stream it is relaying. This log is the ground truth; it is never visible to the observer and is used only to assign phase labels. Flight-phase labels are derived from the logged state by fixed rules established before any model is trained: for example, a multirotor bin is *hover* when the horizontal speed is below 0.7 m/s and the aircraft is airborne, *orbit* when it is executing a loiter-turns command with non-zero ground speed, and *tele-operation* when a manual velocity or stick-override stream is active; the full rule set is included in the released code. Because the labels come from the aircraft and the features come from the network, label and feature are independent by construction.

From packets to windows

The observer's record of a flow is the sequence of its outer packets. We cut it into windows of $T = 128$ bins of 100 ms (12.8 s) with a 2 s stride. A window's label is the majority phase over its bins at the granularity of the task at hand, and windows whose majority phase covers less than 70% of the bins—phase transitions—or that touch a gap in the ground truth are discarded, so that each window has a well-defined phase. To

prevent any flight from contributing windows to more than one split, every sortie—a drone's mission from arming to disarming, with its log upload—is assigned as a whole to the training, validation or test split by a deterministic hash in the proportion 70/15/15; the test windows therefore come from sorties never seen in training, while every site and every phase is represented in each split. Windows that straddle two sorties are discarded. Table 1 summarises the datasets used in this paper and their splits, and Table 2 gives the number of windows per fine phase in each split; the emulated corpus comprises 22,618 labelled windows across the two fleets.

Table 1. Datasets. Source, number of drones and observation sites, captured duration, captured encrypted packets (frames for the public captures), windows in the training / validation / test splits, and number of fine classes. Splits are by sortie (emulated fleets) or by time block (public captures), so no flight or time block appears in two splits.

Dataset	Source	Drones / sites	Hours	Packets	Train	Val.	Test	Classes
SkyNet-A, multirotor (this work)	GNS3, ArduCopter SITL, MAVLink 2 / WireGuard	12 / 4	11.2	1,476,741	7,558	1,518	1,504	7
SkyNet-B, VTOL (this work)	GNS3, ArduPlane SITL, MAVLink 2 / IPsec	12 / 4	11.5	1,754,471	8,031	2,130	1,877	6
PiNcH, steady / moving (public)	real 3DR Solo, ArduCopter, encrypted Wi-Fi [10]	1 / 1	0.33	174,679	239	48	47	2
PiNcH, stick movements (public)	real 3DR Solo, ArduCopter, encrypted Wi-Fi [10]	1 / 1	0.12	66,317	63	21	21	7

Table 2. Windows per fine flight phase and split in the two emulated fleets (12.8 s windows, 2 s stride, majority phase covering at least 70% of the window).

Fleet	Phase	Train	Validation	Test
Multirotor	Ground	2,234	383	460
	Ascent	325	78	56
	Hover	341	69	62
	Cruise	1,830	407	514
	Orbit	884	248	169
	Teleop	1,036	109	64
	Descent	908	224	179
VTOL	Ground	3,374	735	701
	VTOL climb	369	125	99
	VTOL descent	610	183	114
	Cruise	2,093	833	519
	Loiter	1,067	2	251
	Teleop	518	252	193

Every window is presented to the encoder in two complementary views, both computed from the same packets. The *bin view* is the sequence of T per-bin feature vectors: separately for the uplink and downlink directions, a histogram of packet sizes over 41 buckets, the byte count, and the mean, standard deviation and maximum of the inter-arrival times within the bin, giving $d = 90$ features per bin. The *statistics view* is a single vector of 40 window-level statistics of sizes and inter-arrival times per direction (count, mean, standard deviation, median, 10th and 90th percentiles, extremes, byte total, fractions of small and large packets), together with the up/down packet ratio and the packet rate—the features on which earlier UAV traffic analyses were built [9–11]. Every quantity is compressed with $\log(1 + x)$. The statistics summarise the window; the bins preserve its temporal structure. A third, finer view—the sequence of up to 256

individual packets of the window, each described by its size, direction, inter-arrival time and position—was evaluated as well and is reported in the ablation.

Public over-the-air captures

To test the encoder on traffic from a physical aircraft and radio link, we also use the public PiNcH dataset [10]: a 3DR Solo multirotor running ArduCopter, whose encrypted Wi-Fi link to its remote controller was captured passively with a wireless probe. The release provides, for each captured frame, its inter-arrival time, size and direction, together with a label; no payload is available, which matches the threat model. Two labelled subsets are used. The first contains about ten minutes of the drone hovering in place and ten minutes of random movement in all directions (174,679 frames), giving a two-class task, steady versus moving. The second contains about two minutes of each of seven stick movements (pitch down, pitch up, roll left, roll right, throttle up, yaw left, yaw right; 66,317 frames). The frames are converted with the same code that processes our captures into 100 ms bins, window statistics and windows; because the capture is a single continuous session rather than separate sorties, time is cut into blocks of 30 s (15 s for the short movement subset, with 6.4 s windows) and, within each class, the first 70% of the blocks train the model, the next 15% select it and the last 15% are held out, so that no window is evaluated on time it was trained on. The dataset comes from one observer, so DAPPER is trained locally; the federated protocols do not apply.

The DAPPER model

DAPPER maps the two views of a window to a flight phase through four stages—site-local calibration of each view, tokenisation into one sequence, a gated differential-attention encoder with a pooled read-out, and, around them, a path-personalised federated training protocol. The architecture is shown in Figure 3.

Figure 3. DAPPER: data, network and analysis. (a) The three input datasets: the two emulated fleets of this work (SkyNet-A, ArduCopter over WireGuard; SkyNet-B, ArduPlane VTOL over IPsec) and the public PiNcH capture of a real 3DR Solo over encrypted Wi-Fi; each card shows 60 s of uplink bytes per 100 ms with the true state as a coloured strip. (b) Network: site-calibrated bins become 32 patch tokens and the window statistics one statistics token; with a class token and four site prompts they pass through four gated differential-attention blocks (the maps shown are computed by the trained model on a real tele-operation window); only the shared encoder is federated. (c) Class-token attention over that window and the held-out score. (d) One attention head. (e) Which parameters are averaged and which stay at the site. (f) The compared training protocols.

Site-local path calibration

The raw features of a 5G small cell and a geostationary-satellite link differ by orders of magnitude in rate and by their delay structure. Before any shared processing, each view is standardised by the statistics of its own site and then passed through a learnable per-feature affine map,

$$\tilde{x} = \frac{x - \mu_s}{\sigma_s} \odot \gamma_s + \beta_s, \tag{1}$$

where μ_s and σ_s are the mean and standard deviation of the features at site s (estimated once from that site's training windows and frozen), and γ_s, β_s are learned. Because many size buckets are empty almost all of the time at a given site, σ_s is floored

at 0.1 and the standardised value is clipped to $[-10, 10]$, so that a rare burst—a flight-log upload over a satellite link, for instance—cannot drive the attention logits out of range. All four quantities are *personal* to the site and never leave it. Calibration lets the shared encoder see comparable inputs regardless of the enormous path differences, so that the encoder can specialise to the phase signal rather than to the link.

Tokenisation

The calibrated views are turned into one token sequence of width 128. A learnable class token and 4 learnable *path prompts* open the sequence. The statistics view is projected to a single *statistics token*. The bin view is passed through a strided one-dimensional convolution over time (kernel 2×4 bins, stride 4 bins) that produces one *patch token* per 4 bins, to which learned positional embeddings are added; the patches give the encoder a local, 0.4 s-resolution summary of the traffic.

Gated differential-attention encoder

The token sequence is processed by a stack of attention blocks. Standard self-attention places some weight on every position, and in our setting much of that weight lands on tokens dominated by cross-traffic or by link-induced jitter rather than by the C2 cadence—weight that acts as common-mode noise. Differential attention [7] removes it by computing the attention as the difference of two softmax maps,

$$\mathrm{DiffAttn}(X) = \left(\mathrm{softmax}\left(\frac{Q_1 K_1^\top}{\sqrt{d_h}}\right) - \lambda\,\mathrm{softmax}\left(\frac{Q_2 K_2^\top}{\sqrt{d_h}}\right)\right) V, \tag{2}$$

where $Q = [Q_1; Q_2]$ and $K = [K_1; K_2]$ are split into two halves and the scalar

$$\lambda = \exp(\langle \ell_{q1}, \ell_{k1} \rangle) - \exp(\langle \ell_{q2}, \ell_{k2} \rangle) + \lambda_{\mathrm{init}} \tag{3}$$

is learned per layer from vectors $\ell_\cdot$, with $\lambda_{\mathrm{init}} = 0.8 - 0.6\, e^{-0.3(\ell-1)}$ at layer ℓ. The subtraction cancels the component the two maps share and leaves the sparse, differential weight on the tokens that carry the control cadence. Each head output is RMS-normalised, scaled by $(1 - \lambda_{\mathrm{init}})$, and multiplied by a head-specific sigmoid gate computed from the block input, $\sigma(W_g x)$, before the output projection; this output gate [8] lets a head switch itself off for tokens it has nothing to say about. Each block closes with a SwiGLU feed-forward sub-layer under pre-normalisation.

Read-out, path prompts and regularisation

After the final block the class-token embedding is concatenated with the mean of the patch tokens, and a single linear layer maps the result to the C phase logits. The path prompts are n_p learnable vectors, personal to each site, prepended to the sequence; they give the shared attention a small, site-specific context to condition on without changing the shared weights, analogous to prompt tuning. With the calibrations and the prompts personal and everything else shared, DAPPER carries only 780 personal parameters per site out of 0.96 M in total. Because the windows of a one-hour capture overlap heavily, the encoder can memorise flights; training therefore masks a random 15% of the patch tokens of every window (token dropout), applies dropout, label smoothing, small Gaussian noise on both views, and weight decay, and anneals the learning rate with a cosine schedule.

Path-personalised federated training

Training follows a federated protocol over the sites of a fleet. In each round the server broadcasts the current shared encoder; each site loads it, keeps its own personal

parameters, runs several local optimisation steps on its training windows with a class-balanced cross-entropy loss, and returns its updated shared weights. The server averages the shared weights across sites (FedAvg), while the personal parameters—the calibrations and the prompts—are never transmitted or averaged. No raw window ever leaves its site; only shared-encoder weights are exchanged. Algorithm 1 summarises the protocol and the reading of a site that never took part in training.

Algorithm 1 Path-personalised federated training of DAPPER and reading of a new site

Require: sites $\mathcal{S}$, each holding its own labelled windows $\mathcal{D}_s$ (never shared); rounds R; local steps K; batch size B

Ensure: shared encoder θ; personal parameters $\phi_s = (\mu_s, \sigma_s, \gamma_s, \beta_s, P_s)$ for every site s

Server initialises the shared encoder θ (tokenisation, gated differential-attention blocks, read-out)
for all sites $s \in \mathcal{S}$ in parallel **do**
  $\mu_s, \sigma_s \leftarrow$ mean and s.d. of the bin and statistics views over a sample of $\mathcal{D}_s$ {frozen, from unlabelled traffic}
  $\gamma_s \leftarrow \mathbf{1}$, $\beta_s \leftarrow \mathbf{0}$, $P_s \leftarrow$ random prompts
end for
for round $r = 1$ **to** R **do**
  Server broadcasts θ; learning rate η_r follows the cosine schedule
  for all sites $s \in \mathcal{S}$ in parallel **do**
    $\theta_s \leftarrow \theta$
    for step $k = 1$ **to** K **do**
      Sample a batch of B windows from $\mathcal{D}_s$; add Gaussian noise to the bin and statistics views; mask 15% of the patch tokens
      Calibrate each view: $\tilde{x} = \mathrm{clip}\big((x - \mu_s)/\max(\sigma_s, 0.1), \pm 10\big) \odot \gamma_s + \beta_s$
      Build tokens $[\mathrm{CLS}; P_s; \mathrm{stat}; \mathrm{patch}_{1..32}]$ and run the encoder; loss = class-weighted cross-entropy with label smoothing
      Update $(\theta_s, \gamma_s, \beta_s, P_s)$ with AdamW at η_r, gradient norm clipped to 1
    end for
    Site sends only θ_s to the server {ϕ_s stays at the site}
  end for
  Server: $\theta \leftarrow \frac{1}{|\mathcal{S}|} \sum_s \theta_s$ {FedAvg of the shared encoder}
end for
Reading a new site $u \notin \mathcal{S}$: set μ_u, σ_u from its unlabelled traffic, $\gamma_u = \mathbf{1}$, $\beta_u = \mathbf{0}$, $P_u =$ mean of the trained prompts; classify with the frozen θ; optionally fit ϕ_u on a few labelled windows

Experimental protocol

We compare against the standard federated repertoire, all using the same encoder for a fair comparison: *local* (each site trains alone), *central* (a reference that pools all sites with the same total step budget, not available to a distributed observer), *FedAvg* [5], *FedProx* [34], and *FedBN* [37].

To isolate the contribution of each DAPPER component we ablate it against variants of itself only: plain softmax attention in place of the differential map, no output gate, calibration only (no prompts), prompts only (shared calibration), bin patches only, statistics token only, and the model with packet tokens added.

To measure generalisation to a new observation point, we hold one site out of the federation entirely and then read it with the shared encoder, calibrating its personal

parameters on its own unlabelled statistics and, optionally, adapting them on a handful of its labelled windows.

The packet stream of every drone is transformed before any view is built, and DAPPER is retrained on the transformed traffic, so that the adversary adapts to the defence. Four countermeasures are compared: padding every packet up to the next multiple of 256 B or of 512 B; delaying every packet by an independent uniform random 0 ms to 200 ms (timing obfuscation at zero byte cost); and replacing the flow by constant-rate cover traffic of 20 packets of 512 B per second in each direction, which hides sizes and timing entirely. The cost of each is its byte overhead relative to the undefended capture.

Windows use $T = 128$ bins and 40 statistics. The encoder has 4 gated differential-attention layers of width 128 with 4 heads, 4 path prompts, and 0.96 M parameters. Training runs for 20 federated rounds of 50 local steps each, with AdamW at a peak learning rate of 5×10^{-4} annealed by cosine to a tenth over the rounds, batch size 128, label smoothing 0.05, weight decay 0.05, gradient-norm clipping and mixed precision. Every configuration is run for 3 random seeds and we report the mean and standard deviation of the held-out macro-F1. All numbers in this paper are produced directly from the experiment logs by a single script; none are transcribed by hand.

Results

This section reports the recovery of the operational state by each training protocol, the results at finer granularities, the ablation of DAPPER components, the reading of unseen sites, and the effect of traffic-shaping defences.

Recovering the operational state

Table 3 and Figure 4 compare the training protocols on the primary task, all with the same encoder, windows, splits and seeds. The operational state—on the ground, in autonomous flight, or under tele-operation—is recovered almost perfectly from the encrypted stream: DAPPER reaches a macro-F1 of 0.998 on the multirotor fleet and 0.984 on the VTOL fleet (chance 0.33), and the coarsest question, airborne versus on the ground, is answered with 99.8% and 98.1% accuracy (one-versus-rest ROC areas 1.000 and 0.997). On the multirotor fleet every protocol is close to the ceiling. The heterogeneous VTOL fleet, whose sites include LEO and GEO satellite links, separates them: the shared-model protocols FedAvg and FedProx and purely local training trail the methods that keep part of the model at the site, and DAPPER exceeds FedAvg by 2.98 points and comes within half a point of the centralised reference, which pools the raw traffic of every site and is not available to a distributed observer.

Table 3. Held-out macro-F1 by training method (operational state: ground / autonomous flight / tele-operation). Mean ± s.d. over 3 seeds; all methods share the same encoder. Central pools all sites and is a reference not available to a distributed observer.

Method	Multirotor (WireGuard)	VTOL (IPsec)
Local	0.995 ± 0.003	0.933 ± 0.002
Central (reference)	1.000 ± 0.000	0.988 ± 0.001
FedAvg	0.999 ± 0.000	0.954 ± 0.002
FedProx	0.999 ± 0.000	0.959 ± 0.007
FedBN	0.998 ± 0.001	0.983 ± 0.001
DAPPER	0.998 ± 0.001	0.984 ± 0.002

Figure 4. Operational-state recovery by training protocol. Held-out macro-F1, mean and standard deviation over 3 seeds, for the multirotor (left) and VTOL (right) fleets. All protocols share the encoder; Central pools all sites and is a reference only.

Figure 5 reports the macro-F1 per cell site. The confusion matrices in Figure 6 and the one-versus-rest ROC curves in Figure 7 show that the remaining errors sit between the ground and autonomous-flight states around take-off and landing, and that tele-operation is separated almost perfectly, consistent with the 10 Hz manual control stream it adds to the C2 flow. Smoothing the per-window posteriors over five consecutive windows of the same flight gives 0.995 and 0.984 macro-F1 (99.7% and 98.1% accuracy).

Figure 5. Per-site macro-F1. Each protocol at each cell site, averaged over seeds, for the multirotor (left) and VTOL (right) fleets.

Figure 6. Confusion matrices of DAPPER. Row-normalised, held-out flights, operational state; multirotor (left) and VTOL (right) fleets.

Figure 7. Per-state ROC curves of DAPPER. One-versus-rest curves and areas on held-out flights for each fleet.

Finer granularities: where the leak fades

Figure 8 follows the same models down to finer questions. On the four operational phases, which additionally ask whether an airborne aircraft is stationary over a point or in transit, DAPPER reaches 0.838 and 0.757 (Table 4); on the multirotor fleet it is well above FedAvg and above the centralised reference there, whereas on the VTOL fleet all protocols lie within a few points of one another. On the fine kinematic phases DAPPER reaches 0.486 and 0.506 (Table 5). The residual confusions fall almost entirely between hover or orbit and cruise, and between ascent, cruise and descent—phases during which the autopilot streams the same telemetry messages at the same rates, so that the encrypted flow differs only through the sparse mission events (way-point acknowledgements, mode changes, status messages) that happen to fall inside a 12.8 s window. This boundary is a property of the traffic rather than of a training protocol: the local, centralised and personalised federated models all saturate at a similar level on the finer tasks.

Figure 8. Leakage by granularity. Held-out macro-F1 of DAPPER, FedAvg and the centralised reference for the operational state, the operational phases and the fine kinematic phases; mean and standard deviation over 3 seeds.

Table 4. Operational phases (ground / stationary / transit / tele-operation; held-out macro-F1, mean ± s.d. over seeds; chance 0.25).

Method	Multirotor	VTOL
Local	0.811 ± 0.018	0.772 ± 0.045
Central (reference)	0.803 ± 0.014	0.720 ± 0.011
FedAvg	0.743 ± 0.001	0.752 ± 0.021
DAPPER	0.838 ± 0.015	0.757 ± 0.016

Table 5. Fine kinematic phases (seven multirotor / six VTOL phases; held-out macro-F1, mean ± s.d. over seeds where applicable; chance 0.14 / 0.17).

Method	Multirotor	VTOL
Local	0.500 ± 0.008	0.528 ± 0.018
Central (reference)	0.487 ± 0.008	0.489 ± 0.009
FedAvg	0.397 ± 0.003	0.518 ± 0.021
DAPPER	0.486 ± 0.016	0.506 ± 0.009

Ablation

Table 6 and Figure 9 isolate the contribution of each DAPPER component on the operational-state task. One component dominates on both fleets: the temporal bin view. Reading the window through its statistics alone costs about one point on the multirotor fleet and three on the VTOL fleet, so the arrangement of the traffic inside the window carries state information that window-level statistics do not. The attention and personalisation variants, by contrast, all lie within the seed-to-seed variation of the full model on this task: plain softmax attention, no output gate, calibration-only and prompts-only personalisation give the same operational-state accuracy. With the operational state this close to the ceiling, the benefit of personalising the model to each site appears against the shared-model baselines rather than between personalisation variants: every method that keeps part of the model at the site (DAPPER and FedBN) is about three points above FedAvg and FedProx on the VTOL fleet (Table 3). Adding individual packet tokens as a third view lowers the held-out score: the exact packet sequence of a flight is memorised rather than generalised. The convergence curves in Figure 10 show how the federated protocols approach their final accuracy over the rounds.

Table 6. Ablation of DAPPER components (operational state, held-out macro-F1, mean ± s.d. over two seeds; the full model over three).

Variant	Multirotor	VTOL
DAPPER	0.998 ± 0.001	0.984 ± 0.002
DAPPER, softmax attention	0.999 ± 0.001	0.986 ± 0.001
DAPPER, no output gate	0.999 ± 0.001	0.984 ± 0.001
DAPPER, calibration only	0.999 ± 0.000	0.987 ± 0.001
DAPPER, prompts only	0.999 ± 0.001	0.984 ± 0.000
DAPPER, bin patches only (no statistics token)	0.999 ± 0.000	0.983 ± 0.001
DAPPER, statistics token only (no bin patches)	0.988 ± 0.001	0.954 ± 0.000
DAPPER with packet tokens added	0.998 ± 0.000	0.978 ± 0.003

Figure 9. Ablation of DAPPER components. Held-out macro-F1 on the operational state for each variant, both fleets.

Figure 10. Convergence. Validation macro-F1 against federated round for each federated protocol.

Reading an unseen site

Figure 11 holds one site out of the federation entirely—the rural LTE cell of the multirotor fleet and the LEO-satellite cell of the VTOL fleet—and reads it with the frozen shared encoder, calibrating the personal parameters on the site's own unlabelled statistics and optionally adapting them on a few labelled windows. Because a held-out site need not exhibit every state in its test sorties, we report balanced accuracy over the states it does exhibit. Direct transfer, with zero labelled windows, already reads the unseen site almost perfectly (balanced accuracy above 0.99 for DAPPER on both fleets), and a few labelled windows neither help nor hurt it; the FedAvg model, adapted through its classification head, transfers less reliably and can be destabilised by five labelled windows. With the satellite site removed, DAPPER still reaches a macro-F1 above 0.98 on the other VTOL sites, against 0.92–0.93 for FedAvg. The class-token embeddings of held-out windows form state-coherent clusters (Figure 12).

Figure 11. Reading an unseen cell site. Balanced accuracy on the held-out site against the number of labelled windows used to adapt the personal parameters, for DAPPER and FedAvg; bands show the standard deviation over seeds.

Figure 12. Learned embedding. t-SNE of DAPPER class-token embeddings for held-out windows of one site, coloured by true operational state.

Real over-the-air traffic

Table 7 reports DAPPER on the public PiNcH captures of a physical 3DR Solo. Whether the aircraft is hovering or moving is recovered from the encrypted Wi-Fi link with a macro-F1 of 0.964 (96.5% accuracy) on held-out time blocks; this is the Wi-Fi counterpart of the stationary-versus-transit distinction of the emulated fleets and confirms, on real radio traffic, that the encoder reads the state of the aircraft from sizes and timing alone. The seven individual stick movements, for which the release contains two minutes each, are not recovered when the last block of every movement is held out (macro-F1 0.051 on 21 windows): with so little data per class the model memorises the training blocks, and the original study likewise found movement discrimination to depend strongly on the amount of traffic available. The public captures therefore support the coarse-state result and bound, rather than extend, the fine-grained one.

Table 7. DAPPER on the public PiNcH over-the-air captures [10]: a 3DR Solo multirotor running ArduCopter, encrypted Wi-Fi between drone and remote controller, passively captured. Held-out time blocks, macro-F1 and accuracy as mean ± s.d. over three seeds, single observer.

Task	Chance	Windows	Macro-F1	Acc. (%)
Steady vs. moving (12.8 s windows)	0.50	47	0.964 ± 0.010	96.5
Seven stick movements (6.4 s)	0.14	21	0.051 ± 0.004	4.8

Traffic-shaping defences and their cost

Table 8 reports DAPPER retrained on each defended stream, that is, against an adversary that adapts to the defence, together with the byte overhead of the defence. Padding every packet to multiples of 256 B roughly doubles the bytes on the wire and leaves the operational state readable at better than 0.96 macro-F1; padding to multiples of 512 B more than triples them and still leaves above 0.91; randomising every packet's delay by up to 200 ms costs no bandwidth and changes nothing. Only constant-rate cover traffic, which replaces the flow by a fixed cadence of fixed-size packets, reduces the macro-F1 to chance level, at an overhead of about 250%.

Table 8. Traffic-shaping defences: residual leakage versus cost. Held-out macro-F1 of DAPPER retrained on the defended traffic (operational state, seed 1) and the byte overhead of each countermeasure relative to the undefended capture.

	Multirotor		VTOL	
Defence	Macro-F1	Overhead	Macro-F1	Overhead
None (undefended)	0.998	0%	0.984	0%
Pad to multiples of 256 B	0.995	+78%	0.964	+97%
Pad to multiples of 512 B	0.979	+230%	0.913	+278%
Random delay 0 ms to 200 ms	0.999	+0%	0.984	+0%
Constant-rate cover traffic	0.127	+254%	0.236	+253%

Comparison with prior studies

Table 9 places this work beside the published studies of drone traffic analysis. The figures of the earlier studies are those reported by their authors on their own captures and are not directly comparable with ours, because the links, aircraft, states and protocols differ; the table is a map of what each study observes and resolves. The earlier work eavesdrops the direct Wi-Fi link between a drone and its controller at a single capture point and resolves presence, a flying-or-at-rest status, two or three operation modes, or the drone model. This work observes a tunnelled cellular C2 path shared with other users, resolves the aircraft state at three granularities, trains across eight heterogeneous observation points without pooling their captures, reads sites it never trained on, and quantifies the defences. On the one public capture that overlaps with the earlier settings (PiNcH, hovering versus moving), the encoder reaches a macro-F1 of 0.964.

Table 9. Drone traffic-analysis studies. Observed link, aircraft, what is resolved, evaluation setting and the headline figure reported by each study on its own data. OTA = over the air; FL = federated learning.

Study	Link observed	Aircraft	Resolved	Setting	Reported result
Sciancalepore et al. 2019 [9]	Wi-Fi drone–RC, OTA	3DR Solo (ArduCopter)	flying vs. at rest	one capture point	status identified from inter-arrival times; delay below 4 s at 93% accuracy
PiNcH 2020 [10]	Wi-Fi drone–RC, OTA	3DR Solo (ArduCopter)	presence, steady vs. moving, 7 stick movements	one capture point, 6 environments	detection rate $\geq$ 97% up to 200 m; movement discrimination strongly sample-size dependent
Alipour-Fanid et al. 2020 [11]	Wi-Fi drone–RC, OTA	8 consumer UAV types	UAV detection; operation modes	one capture point	detection 85.7–95.2% within 0.15–0.35 s; operation modes 88.5–98.2%
Liang and Dong 2024 [12]	Wi-Fi, OTA	consumer drones	drone model	one capture point	model identification from encrypted traffic
Nassi et al. 2021 [13]	FPV video stream, OTA	consumer drone	whether a target is being filmed	one capture point	time-domain analysis of the encrypted stream
This work	MAVLink 2 in WireGuard / IPsec over emulated cellular sites (5G, LTE, LEO, GEO) with cross-traffic	12 ArduCopter multirotors, 12 ArduPlane VTOL (SITL)	operational state (3), operational phases (4), kinematic phases (7 / 6); defences	8 observation points, FL without pooling, held-out flights and held-out sites	macro-F1 0.998 / 0.984 (state), 0.838 / 0.757 (phases), 0.486 / 0.506 (kinematic); PiNcH steady vs. moving 0.964

Discussion

Encrypting the C2 link of a cellular-connected drone did not conceal its operational state from a passive on-path observer. On held-out flights, whether the aircraft was on the ground, in autonomous flight or under manual control was recovered with a macro-F1 of 0.998 for the multirotor fleet over WireGuard and 0.984 for the VTOL fleet over IPsec, at sites ranging from 5G to geostationary satellite backhaul. Earlier studies reported comparable separability for two or three states on a direct Wi-Fi link between drone and controller [9, 11]; the present results show that the same information survives a tunnelled cellular path with shared backhaul, cross-traffic and link-dependent delay, and that it is available to an observer who sees only the outer packets of the tunnel.

The leak was not uniform across granularities. It was strongest for tele-operation, during which the ground station streams velocity set-points at a fixed 10 Hz cadence, and for the ground state, during which the telemetry stream and the post-flight log upload differ markedly from flight. Within autonomous flight, the four operational phases were recovered with a macro-F1 of 0.838 and 0.757 and the seven or six kinematic phases with 0.486 and 0.506. ArduPilot streams position, attitude and status messages at rates fixed by the ground station irrespective of whether the aircraft is hovering, cruising or orbiting, so these phases differ in the encrypted flow only through sparse mission events—way-point acknowledgements, mode changes and status messages—that fall inside a given window. All training protocols, including centralised training, reached similar levels on the finer tasks, which indicates that this limit originates in the traffic rather than in the learner.

The federated results show where personalisation matters. On the multirotor fleet, whose four sites are terrestrial cells, all protocols reached the ceiling on the operational state. On the VTOL fleet, which includes LEO and GEO satellite sites, purely local training reached 0.933 and FedAvg 0.954, whereas the protocols that keep part of the model at each site reached 0.983 (FedBN) and 0.984 (DAPPER), against 0.988 for

centralised training on pooled data. On the operational phases of the multirotor fleet, DAPPER reached 0.838 against 0.743 for FedAvg and 0.803 for centralised training. The ablation attributes the gain to two elements: the temporal bin view, without which the macro-F1 fell to 0.988 and 0.954, and the site-personal parameters. Replacing differential by softmax attention, or removing the output gate, did not change the operational-state accuracy within the seed-to-seed variation. The site-personal calibration also allowed a held-out cell site to be read without labelled data from that site, which is relevant to an observer extending coverage to new locations.

The defence experiments indicate that conventional padding is insufficient for drone C2. Padding packets to multiples of 256 B or 512 B increased the transmitted bytes by up to about 280% and left the operational state recoverable with a macro-F1 above 0.91, and random delays of up to 200 ms had no measurable effect, because the discriminating information lies in the number and size class of packets per unit time rather than in their exact timing. Only constant-rate cover traffic reduced the adversary to chance, at an overhead of about 250% for the multirotor and VTOL fleets. This agrees with findings for encrypted DNS and Wi-Fi device fingerprinting, where effective shaping also required substantial overhead [45, 46]. For UAV operators, the result implies that the confidentiality of operational state must be addressed at the traffic-shaping layer, in addition to the cipher, and budgeted against the available uplink capacity.

Limitations

The traffic was generated in emulation. The ArduPilot flight stack, the encrypted tunnels and the shaped links with cross-traffic reproduce the timing structure of a real autopilot, but radio-layer effects of a physical cellular network—scheduling, retransmission and handover—are approximated by `netem` rather than reproduced, and over-the-air captures are required to confirm the absolute leakage figures. The phase taxonomies are design choices; reporting three granularities bounds their influence. The ground-control software and companion applications were implemented for this study, and a different telemetry configuration would change the absolute numbers. Validation and test splits contain a limited number of sorties per site, so differences of one or two points between variants are within the seed-to-seed variation. Finally, a single target flow was analysed; correlating several flows, or combining traffic with coarse location, would strengthen an adversary.

Conclusion

In this paper, we have studied the problem of identifying the flight phase of a cellular-connected UAV from its encrypted C2 traffic. We have shown that a passive observer who sees only the size, direction and timing of the encrypted packets can determine whether the aircraft is on the ground, in autonomous flight or under manual control, with a macro-F1 of 0.998 and 0.984 on two fleets of aircraft, two encrypted transports and link conditions ranging from 5G to satellite. The accuracy decreases for finer-grained phases, since the telemetry is sent at a fixed rate and only sparse mission events distinguish one phase from another. We have proposed DAPPER, a federated learning framework based on a gated differential-attention encoder with site-specific calibration and prompts. The proposed method achieves results within 0.5 points of a centralised model without exchanging any traffic between sites, outperforms the shared-model protocols FedAvg and FedProx on the heterogeneous satellite-backed fleet, and can be applied to cell sites that were not seen during training. We have also evaluated three traffic-shaping countermeasures: packet padding and random delays were found to be ineffective, whereas constant-rate cover traffic removed the leakage at

a bandwidth overhead of about 250%. The results were confirmed on the public PiNcH dataset collected from a real drone. The GNS3 testbed regenerates the complete dataset with real flight firmware and real encrypted tunnels, and it has been released together with the source code so that other countermeasures and traffic profiles can be evaluated. In future work, we plan to validate the results on over-the-air captures from a commercial cellular network, to extend the analysis to several concurrent flows and to video streams, and to design cover-traffic schemes with lower bandwidth cost.

Data availability

The processed dataset used in this study—for each of the 24 drones, the per-packet records of the encrypted C2 flow (timestamp, outer IP length and direction), the 100 ms bin features, the per-bin flight-phase labels and the mission index—and the result files of all reported runs are publicly available at `https://github.com/kmkholm/DAPPER-UAV`. No packet payloads were captured. The public PiNcH drone-traffic dataset [10] used in the over-the-air experiment is available from its authors at `https://cri-lab.net/pinch`.

Code availability

The complete source code—the GNS3 testbed (Docker image, node roles, mission supervisor, capture and ground-truth logging), packet processing, the DAPPER model, federated training, the traffic-shaping defences, and the scripts that generate every table and figure of this paper—is publicly available under the MIT license at `https://github.com/kmkholm/DAPPER-UAV`.

References

1. Shrestha R, Omidkar A, Roudi SA, Abbas R, Kim S. Machine-Learning-Enabled Intrusion Detection System for Cellular Connected UAV Networks. Electronics. 2021;10(13):1549. doi:10.3390/electronics10131549.

2. Dang Y, Karakoc A, Norshahida S, Jäntti R. 3D Radio Map-Based GPS Spoofing Detection and Mitigation for Cellular-Connected UAVs. IEEE Transactions on Machine Learning in Communications and Networking. 2023;1:313–327. doi:10.1109/tmlcn.2023.3316150.

3. Dixit B, Ananthapadmanabhan A, Thahsin A, Pathak S, Kasbekar GS, Maity A. A Novel Cipher for Enhancing MAVLink Security: Design, Security Analysis, and Performance Evaluation Using a Drone Testbed. IEEE Open Journal of the Communications Society. 2025;6:9027–9051. doi:10.1109/ojcoms.2025.3621318.

4. Mak B, Yang J, Chen J, Ratazzi P, Wang Y. Symbolic and Safety-Centric Formal Verification of MAVLink for Autonomous eVTOL Systems. IEEE Networking Letters. 2025;7(4):260–264. doi:10.1109/lnet.2025.3610580.

5. McMahan B, Moore E, Ramage D, Hampson S, Agüera y Arcas B. Communication-Efficient Learning of Deep Networks from Decentralized Data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS); 2017. p. 1273–1282.

6. Ariffin A, Afifi F, Zaki F, Hanif H, Anuar NB. Federated learning in network traffic classification: Taxonomy of implementation, application, and impact on sixth-generation wireless networks. Engineering Applications of Artificial Intelligence. 2025;158:111471. doi:10.1016/j.engappai.2025.111471.

7. Ye T, Dong L, Xia Y, Sun Y, Zhu Y, Huang G, et al. Differential Transformer. In: 13th International Conference on Learning Representations (ICLR); 2025. p. 65910–65930.

8. Qiu Z, Wang Z, Zheng B, Huang Z, Wen K, Yang S, et al. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. In: Advances in Neural Information Processing Systems. vol. 38; 2025. p. 110931–110957.

9. Sciancalepore S, Ibrahim OA, Oligeri G, Di Pietro R. Detecting Drones Status via Encrypted Traffic Analysis. In: Proceedings of the ACM Workshop on Wireless Security and Machine Learning; 2019. p. 67–72. doi:10.1145/3324921.3328791.

10. Sciancalepore S, Ibrahim OA, Oligeri G, Di Pietro R. PiNcH: An effective, efficient, and robust solution to drone detection via network traffic analysis. Computer Networks. 2020;168:107044. doi:10.1016/j.comnet.2019.107044.

11. Alipour-Fanid A, Dabaghchian M, Wang N, Wang P, Zhao L, Zeng K. Machine Learning-Based Delay-Aware UAV Detection and Operation Mode Identification over Encrypted Wi-Fi Traffic. IEEE Transactions on Information Forensics and Security. 2020;15:2346–2360. doi:10.1109/tifs.2019.2959899.

12. Liang D, Dong Y. Identifying Consumer Drones via Encrypted Traffic. In: IUTAM Bookseries. vol. 40; 2024. p. 65–83. doi:10.1007/978-3-031-39303-7_5.

13. Nassi B, Ben-Netanel R, Shamir A, Elovici Y. Game of Drones - Detecting Spying Drones Using Time Domain Analysis. In: Cyber Security Cryptography and Machine Learning; 2021. p. 128–144. doi:10.1007/978-3-030-78086-9_10.

14. Bayrak S. Unveiling intrusions: explainable SVM approaches for addressing encrypted Wi-Fi traffic in UAV networks. Knowledge and Information Systems. 2024;66(11):6675–6695. doi:10.1007/s10115-024-02181-9.

15. Wang H, Yang Z, Park S, Yang Y, Kim S, Lunardi W, et al. SoundBoost: Effective RCA and Attack Detection for UAV via Acoustic Side-Channel. In: 2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN); 2025. p. 289–302. doi:10.1109/dsn64029.2025.00039.

16. Hamza MA, Mohsin M, Khalil M, Kazam Abbas Kazmi SM. MAVLink Protocol: A Survey of Security Threats and Countermeasures. In: 2024 4th International Conference on Digital Futures and Transformative Technologies (ICoDT2); 2024. p. 1–8. doi:10.1109/icodt262145.2024.10740195.

17. Sabuwala N, Daruwala RD. Securing Unmanned Aerial Vehicles by Encrypting MAVLink Protocol. In: 2022 IEEE Bombay Section Signature Conference (IBSSC); 2022. p. 1–6. doi:10.1109/ibssc56953.2022.10037546.

18. Chen Z, Cao Z, Yu F, Yuan X, Zhu H. Mission-preserving protocol defense: An in-situ MAVLink honeypot with benchmark-calibrated thresholds and constant-space logging. Computer Communications. 2026;257:108613. doi:10.1016/j.comcom.2026.108613.

19. Colton Z, Oracevic A, Dilek S. Timestamp Manipulation-Based GPS Spoofing Attacks on the MAVLink 2.0 Protocol for UAV Communication: An Empirical Study. IEEE Transactions on Communications. 2026;74:2564–2579. doi:10.1109/tcomm.2025.3644474.

20. Abdalla AS, Marojevic V. Security Threats and Cellular Network Procedures for Unmanned Aircraft Systems: Challenges and Opportunities. IEEE Communications Standards Magazine. 2022;6(4):104–111. doi:10.1109/mcomstd.0003.2100108.

21. Veksler M, Akkaya K, Uluagac S. Catch Me if You Can: Covert Information Leakage from Drones using MAVLink Protocol. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security; 2024. p. 902–914. doi:10.1145/3634737.3637672.

22. Abu Al-Haija Q, Al Badawi A. High-performance intrusion detection system for networked UAVs via deep learning. Neural Computing and Applications. 2022;34(13):10885–10900. doi:10.1007/s00521-022-07015-9.

23. Li X, Zhang T, Wang J, Nie J, Wang J, Wu X, et al. Enhance UAV Network Resilience by Malicious Traffic Detection: A Twin Graph Encoder Approach. IEEE Transactions on Communications. 2026;74:9138–9153. doi:10.1109/tcomm.2026.3693173.

24. Shakkah M, Al-sellami B, A Hagar A, Tawfik M. MSDA-GDS: A Dual-Branch Hybrid Federated Explainable Deep Learning Framework for CAN Bus Intrusion Detection in Internet of Vehicles. Statistics, Optimization & Information Computing. 2026;15(5):4437–4463. doi:10.19139/soic-2310-5070-3599.

25. H Abdelhaliem A, Bajeszeyadaljunaeidia, S Fathi I, Tawfik M, Alsmadi I. Integrating Fractional Generalized Laguerre Moment with Deep Residual Learning for Real-Time Attack Classification. International Journal of Wireless and Microwave Technologies. 2026;16(3):233–253. doi:10.5815/ijwmt.2026.03.16.

26. Tawfik M, Tarazi H, Dalalah A, Bajeszeyadaljunaeidia, Shaban WM, Fathi IS. Few-shot android malware classification with quantum-enhanced prototypical learning and drift detection. Scientific Reports. 2026;16(1):10744. doi:10.1038/s41598-026-45738-0.

27. Abushariah AAM, Tawfik M, Al-sellami B, Sawah MS. Multistage advanced persistent threat detection and malware attribution using Mamba state space models with Kolmogorov Arnold networks. Scientific Reports. 2026;16(1):30072. doi:10.1038/s41598-026-71232-8.

28. Papadogiannaki E, Ioannidis S. A Survey on Encrypted Network Traffic Analysis Applications, Techniques, and Countermeasures. ACM Computing Surveys. 2022;54(6):1–35. doi:10.1145/3457904.

29. Sirinam P, Imani M, Juarez M, Wright M. Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security; 2018. p. 1928–1943. doi:10.1145/3243734.3243768.

30. Sirinam P, Mathews N, Rahman MS, Wright M. Triplet Fingerprinting: More Practical and Portable Website Fingerprinting with N-shot Learning. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security; 2019. p. 1131–1148. doi:10.1145/3319535.3354217.

31. Mathews N, Holland JK, Hopper N, Wright M. Laserbeak: Evolving Website Fingerprinting Attacks with Attention and Multi-Channel Feature Representation. IEEE Transactions on Information Forensics and Security. 2024;19:9285–9300. doi:10.1109/tifs.2024.3468171.

32. Li X, Shi C, Xu N, Su Y, Kaup A, Liu D, et al. Position-Aware Differential Denoising Transformer for Semantic Segmentation of Remote Sensing Images. IEEE Geoscience and Remote Sensing Letters. 2026;23:1–5. doi:10.1109/lgrs.2025.3649407.

33. Ahmad M, Mazzara M, Distefano S, Khan AM, Ullo SL. DiffFormer: A Differential Spatial-Spectral Transformer for Hyperspectral Image Classification. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing. 2025;18:10419–10428. doi:10.1109/jstars.2025.3558889.

34. Li T, Sahu AK, Talwalkar A, Smith V. Federated Learning: Challenges, Methods, and Future Directions. IEEE Signal Processing Magazine. 2020;37(3):50–60. doi:10.1109/MSP.2020.2975749.

35. Tan AZ, Yu H, Cui L, Yang Q. Towards Personalized Federated Learning. IEEE Transactions on Neural Networks and Learning Systems. 2023;34(12):9587–9603. doi:10.1109/tnnls.2022.3160699.

36. Sabah F, Chen Y, Yang Z, Azam M, Ahmad N, Sarwar R. Model optimization techniques in personalized federated learning: A survey. Expert Systems with Applications. 2024;243:122874. doi:10.1016/j.eswa.2023.122874.

37. Li X, Jiang M, Zhang X, Kamp M, Dou Q. FedBN: Federated Learning on Non-IID Features via Local Batch Normalization. In: 9th International Conference on Learning Representations (ICLR); 2021. .

38. Tawfik M, Obeidat RA, Kamel S, Aljarrah NA, Shehadeh HH, Dalalah A. FedEmoNet: Privacy-preserving federated learning with TCN-Transformer fusion for cross-corpus speech emotion recognition. PLOS One. 2026;21(5):e0342953. doi:10.1371/journal.pone.0342953.

39. Tawik M, Heshmat M, Fathi IS, Shaban AS, Shakah G, Shaban WM. Federated learning-based cervical cancer classification using a novel hybrid KAN-ViT-autoencoder architecture. Neural Computing and Applications. 2026;38(14):588. doi:10.1007/s00521-026-12311-9.

40. Naz A, Ullah I, Uzair M, Khokhar MF, Sabir A, Khan RU. AFL-SecNet: An adaptive federated learning framework for secure and privacy-preserving network traffic analysis. Peer-to-Peer Networking and Applications. 2026;19(1):25. doi:10.1007/s12083-025-02155-w.

41. Hussein SA, Répás SR. Impact of Server-Side Aggregation on Federated Traffic Classification Under Heterogeneous Data Distributions. Big Data and Cognitive Computing. 2026;10(6):167. doi:10.3390/bdcc10060167.

42. Wei Y, Cheng G, Qin T, Chen Z. A Hypernetwork-based Personalized Federated Learning Framework for Encrypted Traffic Classification. In: 2023 19th International Conference on Mobility, Sensing and Networking (MSN); 2023. p. 536–543. doi:10.1109/msn60784.2023.00082.

43. Guan X, Du R, Wang X, Qu H. A Personalized Federated Multi-task Learning Scheme for Encrypted Traffic Classification. In: Artificial Neural Networks and Machine Learning – ICANN 2023; 2023. p. 258–270. doi:10.1007/978-3-031-44213-1_22.

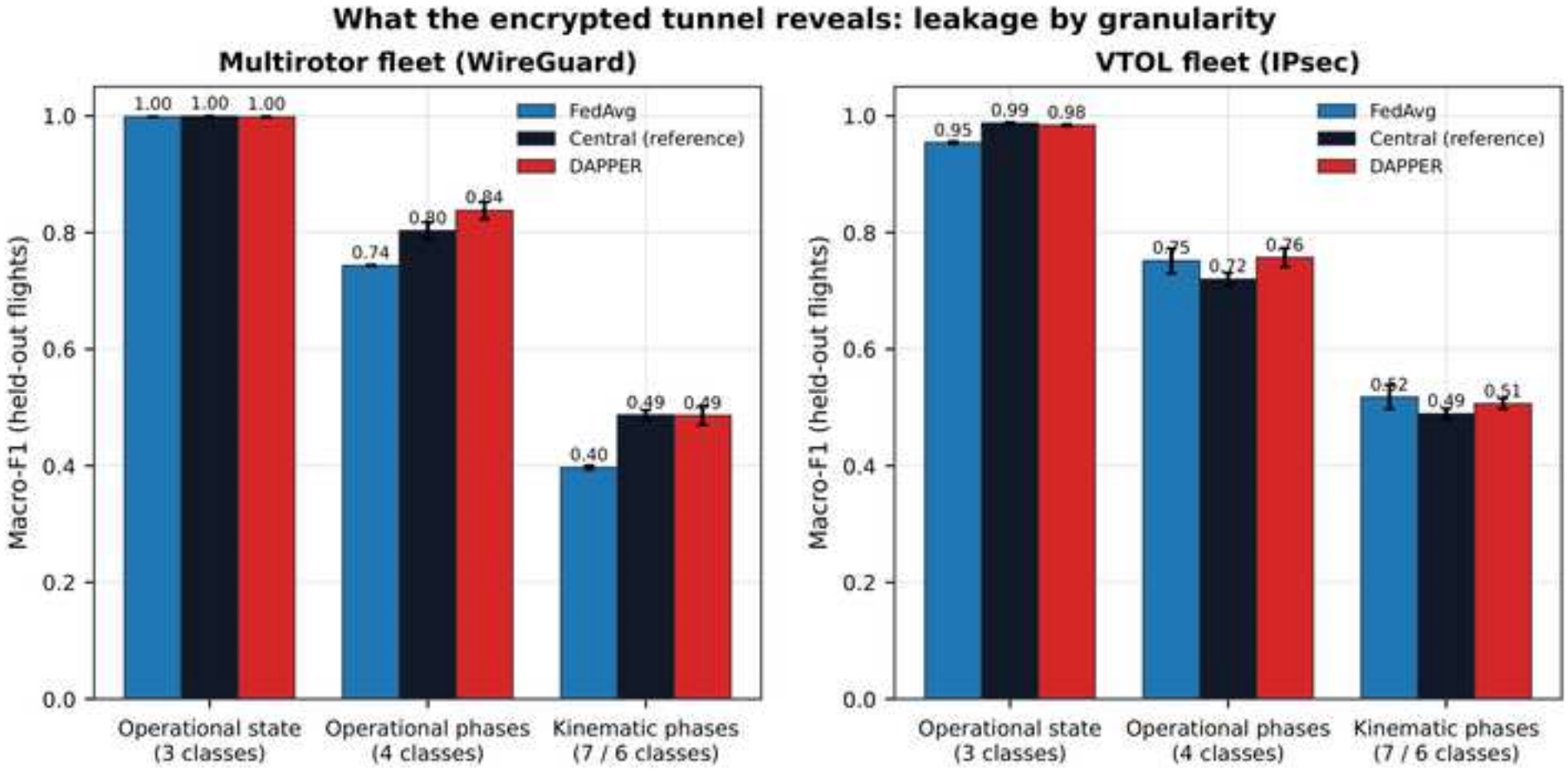
44. Fardous S, Sharif K, Li F, Manjotho AA, Zhu L. DAIR-FedMoE: Hierarchical MoE for Federated Encrypted Traffic Classification Under Compound Drift. IEEE Transactions on Dependable and Secure Computing. 2026;23(4):7709–7726. doi:10.1109/tdsc.2026.3676447.

45. Niakanlahiji A, Orlowski S, Vahid A, Jafarian JH. Toward practical defense against traffic analysis attacks on encrypted DNS traffic. Computers & Security. 2023;124:103001. doi:10.1016/j.cose.2022.103001.

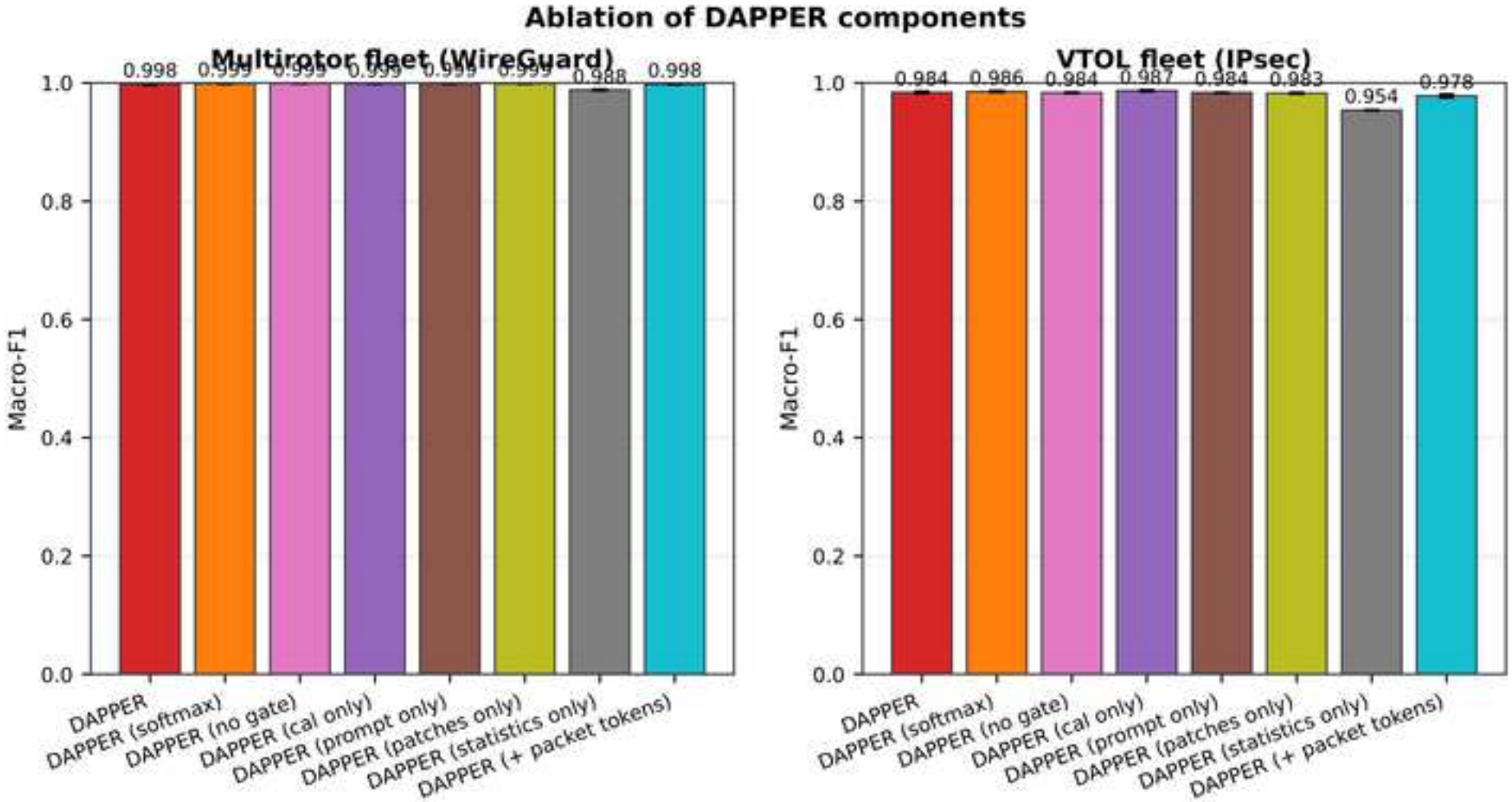
46. Alyami M, Alkhowaiter M, Ghanim MA, Zou C, Solihin Y. MAC-Layer Traffic Shaping Defense Against WiFi Device Fingerprinting Attacks. In: 2022 IEEE Symposium on Computers and Communications (ISCC); 2022. p. 1–7. doi:10.1109/iscc55528.2022.9913056.

47. Pikov V, Ryapukhin A, Veas Iniesta D. Information Protection in Complexes with Unmanned Aerial Vehicles Using Moving Target Technology. Inventions. 2023;8(1):18. doi:10.3390/inventions8010018.

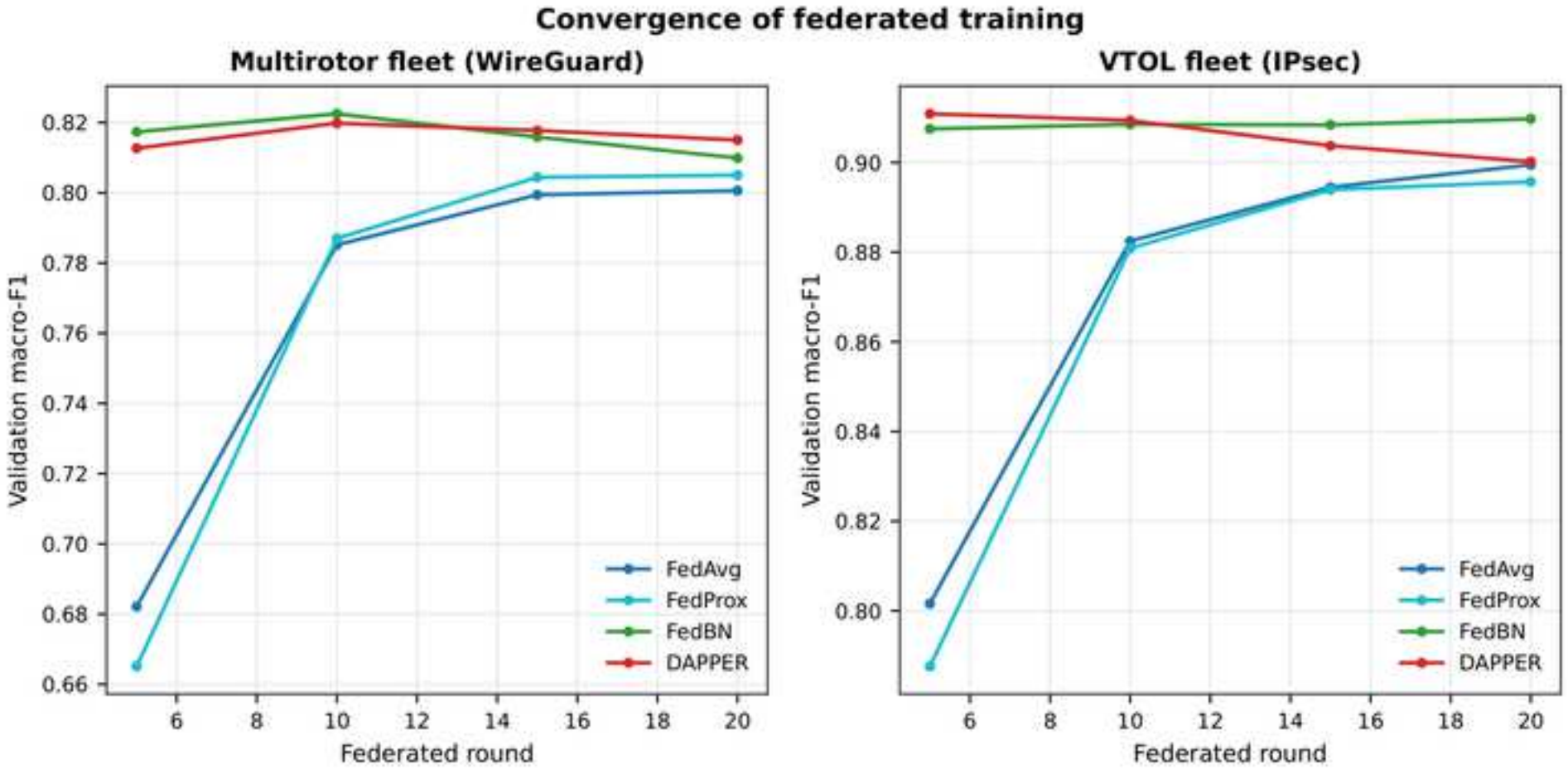
Click here to access/download;Figure;Fig8.tif



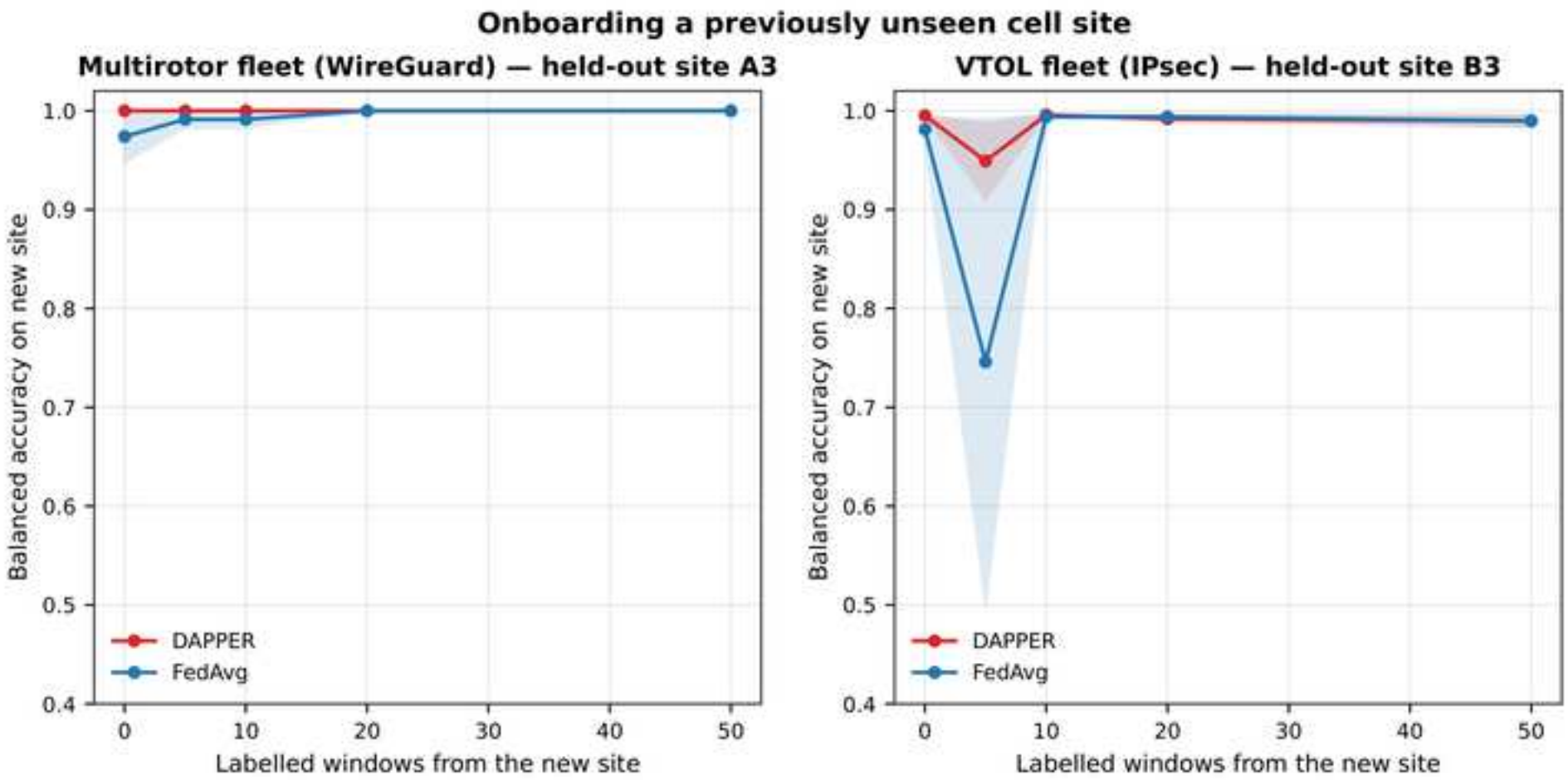
Click here to access/download;Figure;Fig9.tif



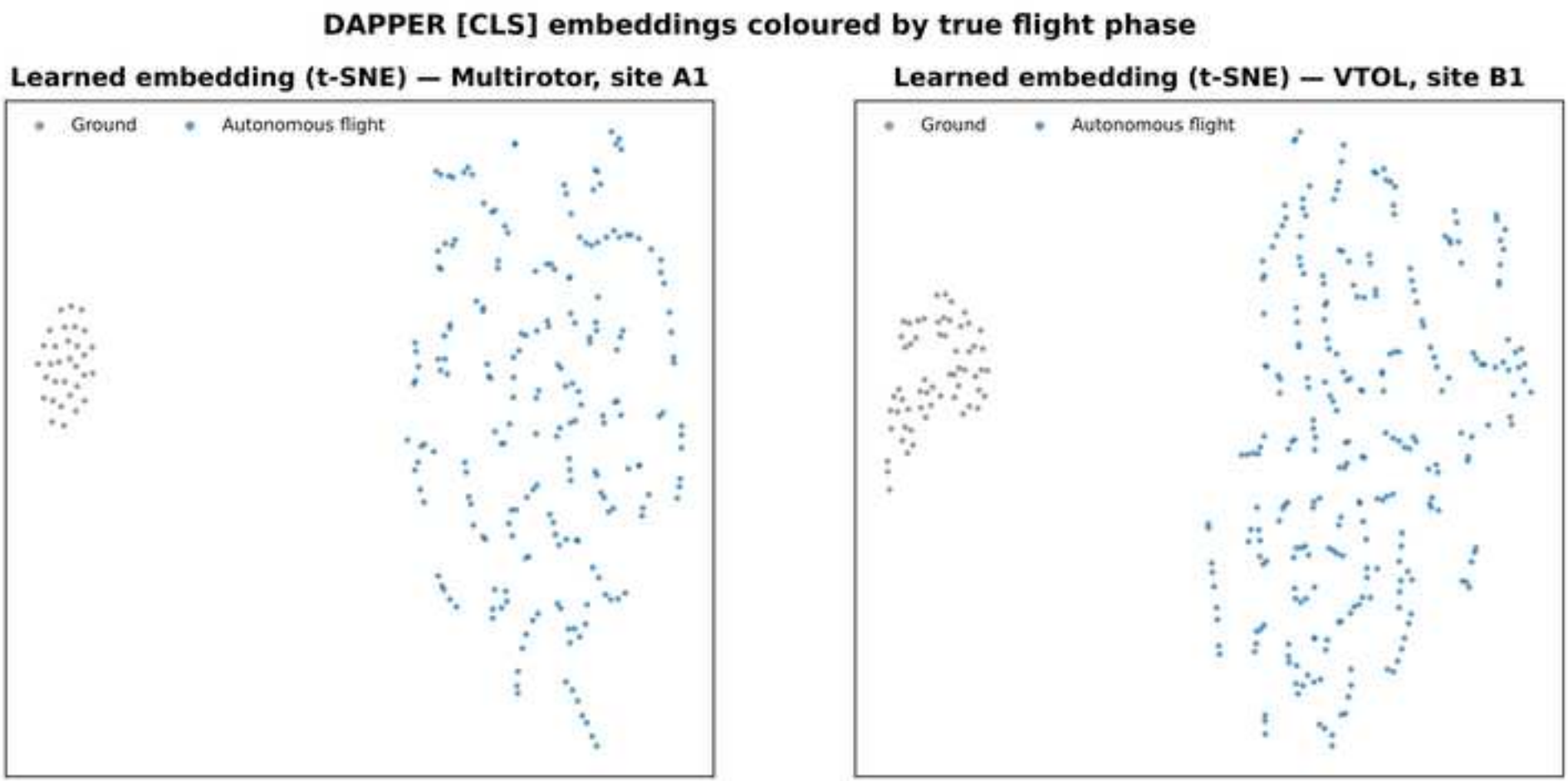
Click here to access/download;Figure;Fig10.tif



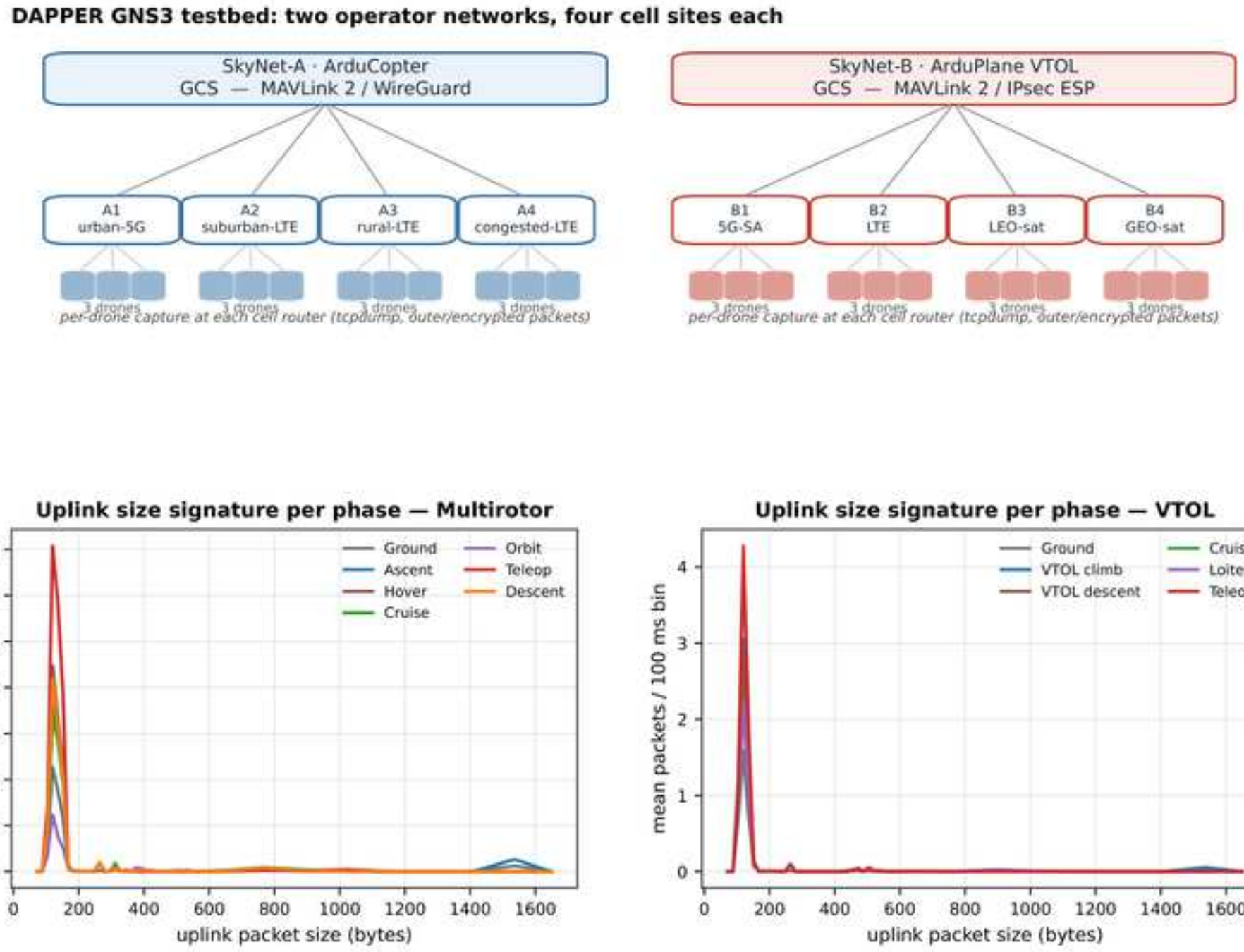
Click here to access/download;Figure;Fig11.tif



Click here to access/download;Figure;Fig12.tif

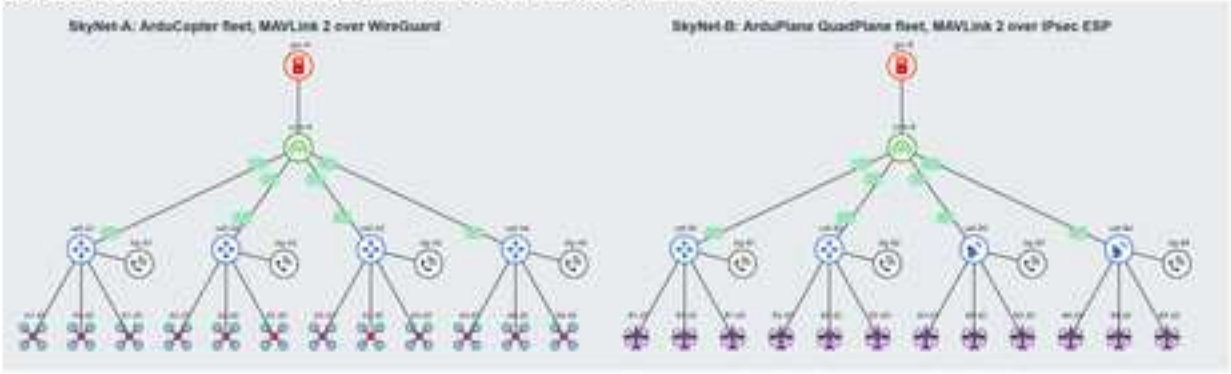


Click here to access/download;Figure;Fig1.tif



Click here to access/download;Figure;Fig2.tif

(a) GNS3 project map: two fleets, eight cell sites, 24 drones, all 44 nodes running



(b) multirotor A2-d1 in flight: ArduCopter SITL state



(c) VTOL B3-d1 in flight behind the LEO site



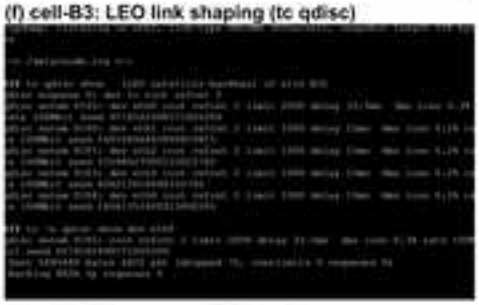
(d) A1-d1: WireGuard tunnel (wg show)



(e) B1-d1: IPsec ESP SAs (ip xfrm state)



(f) cell-B3: LEO link shaping (tc qdisc)



(g) cell-A1: encrypted WireGuard packets



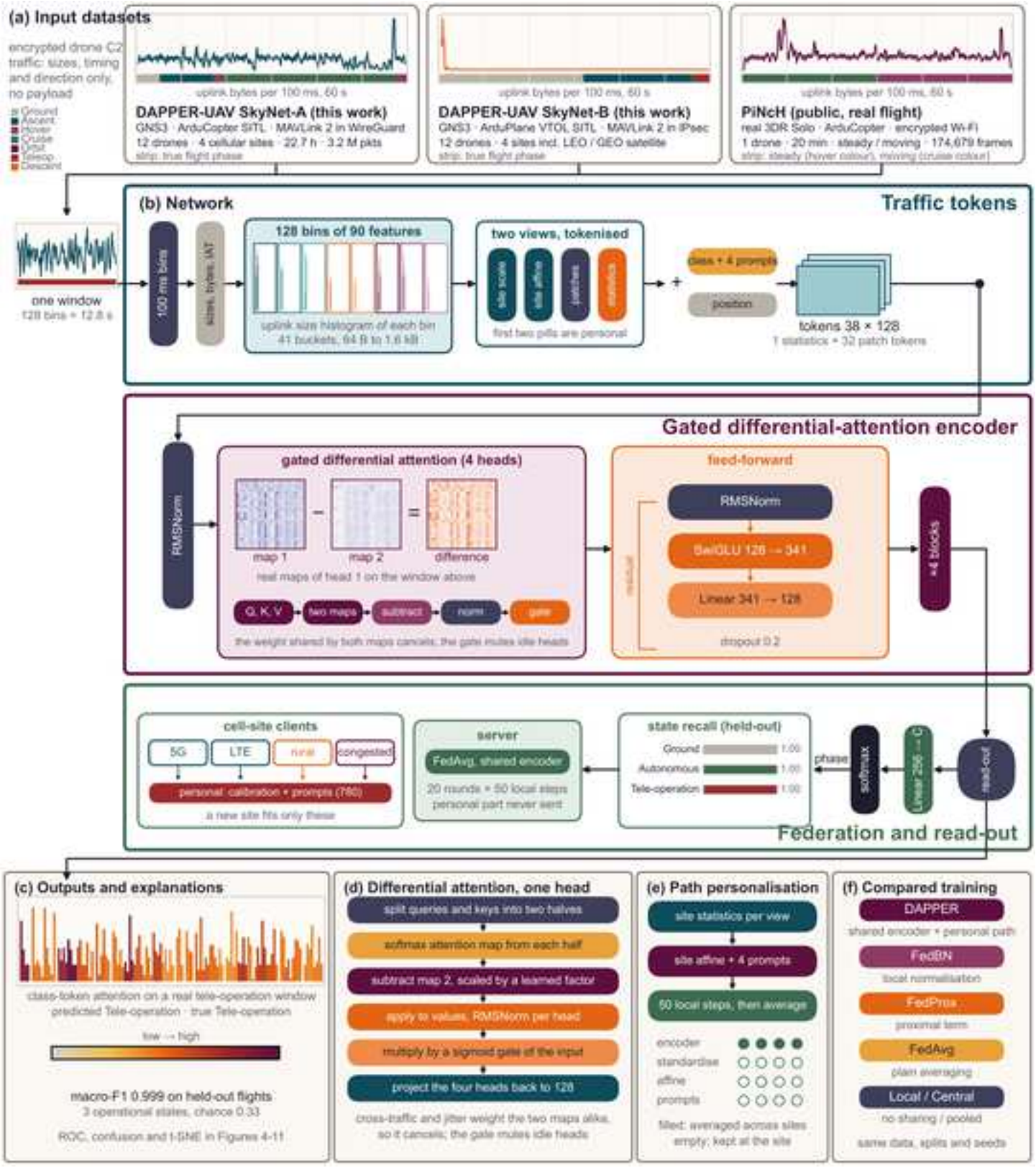
(h) cell-B1: encrypted ESP packets



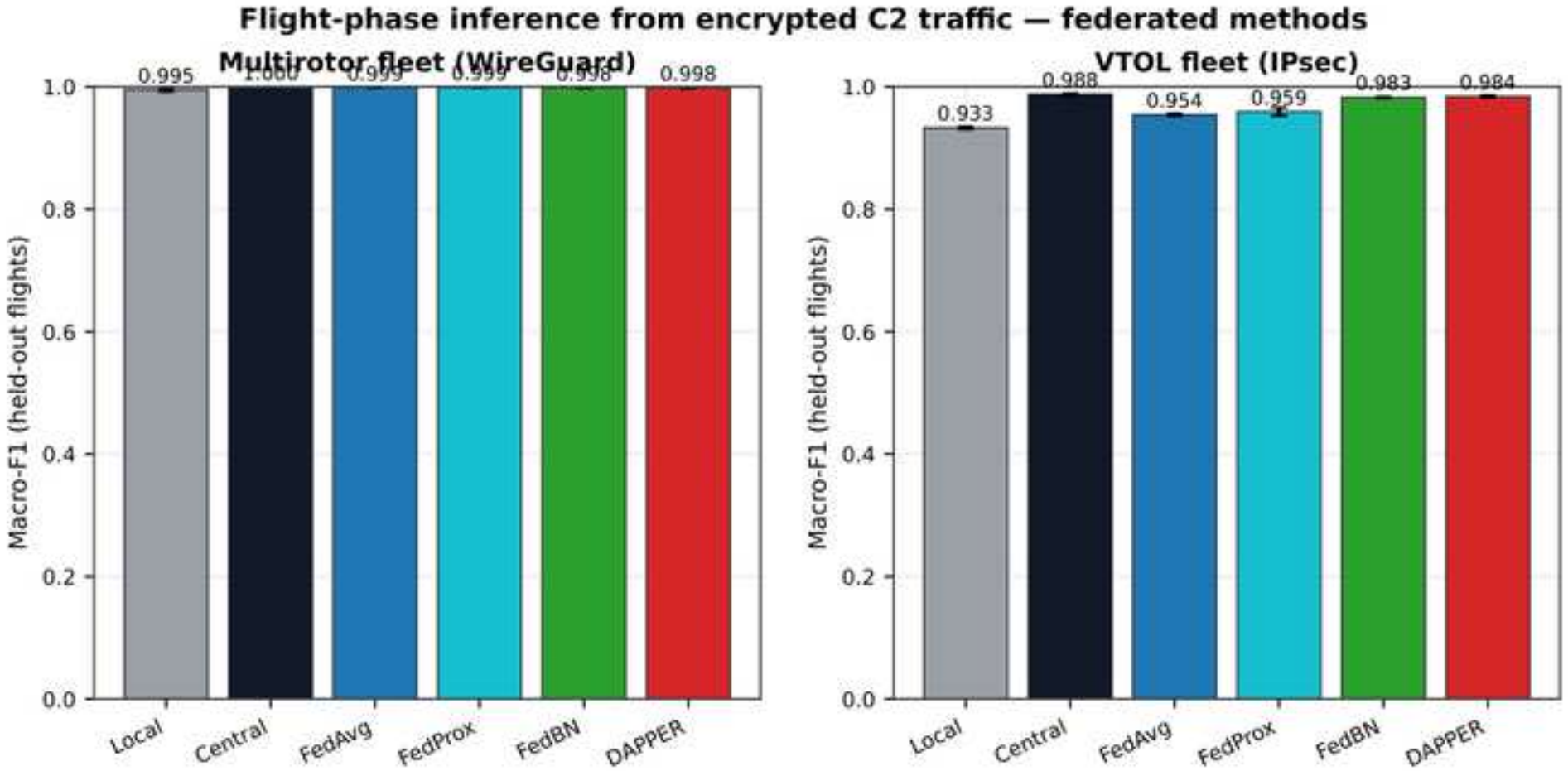
(i) ground-control station A: mission events



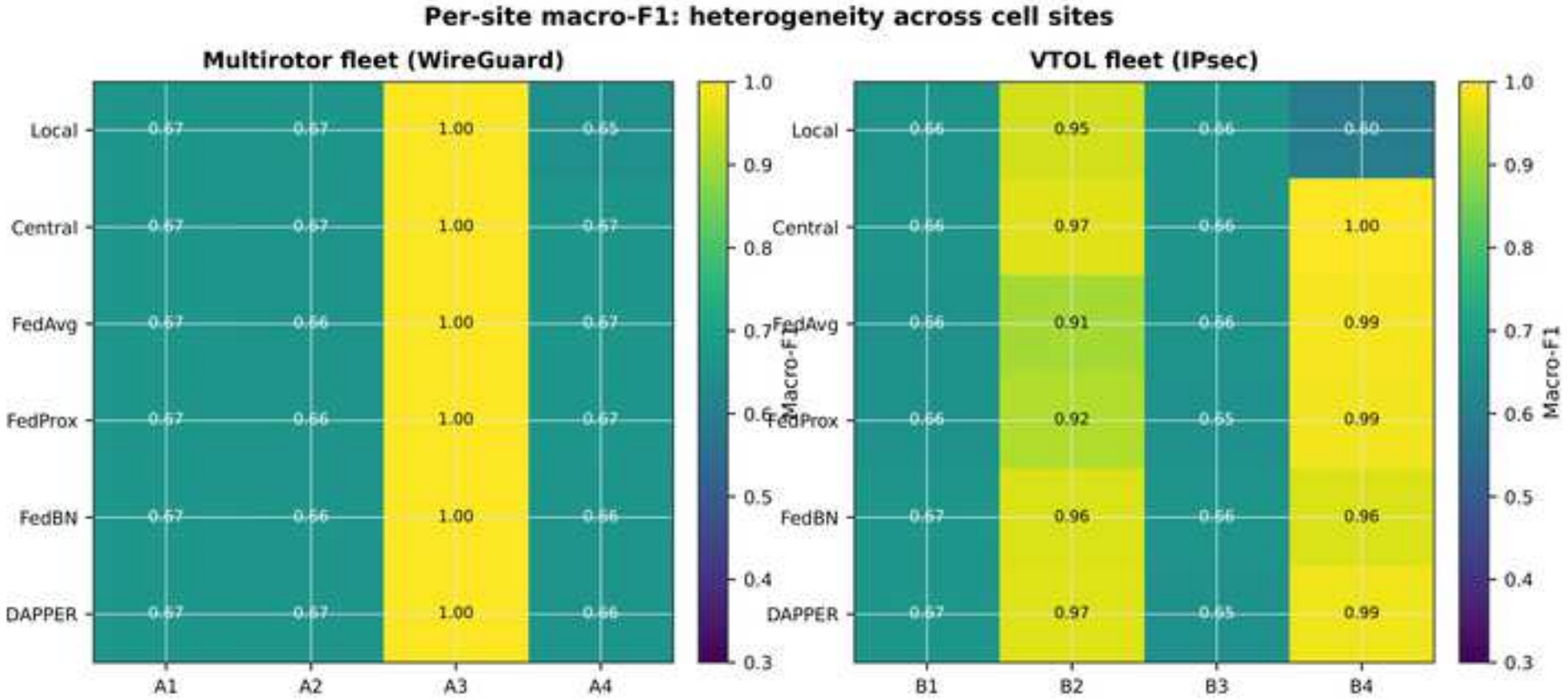
Click here to access/download;Figure;Fig3.tif



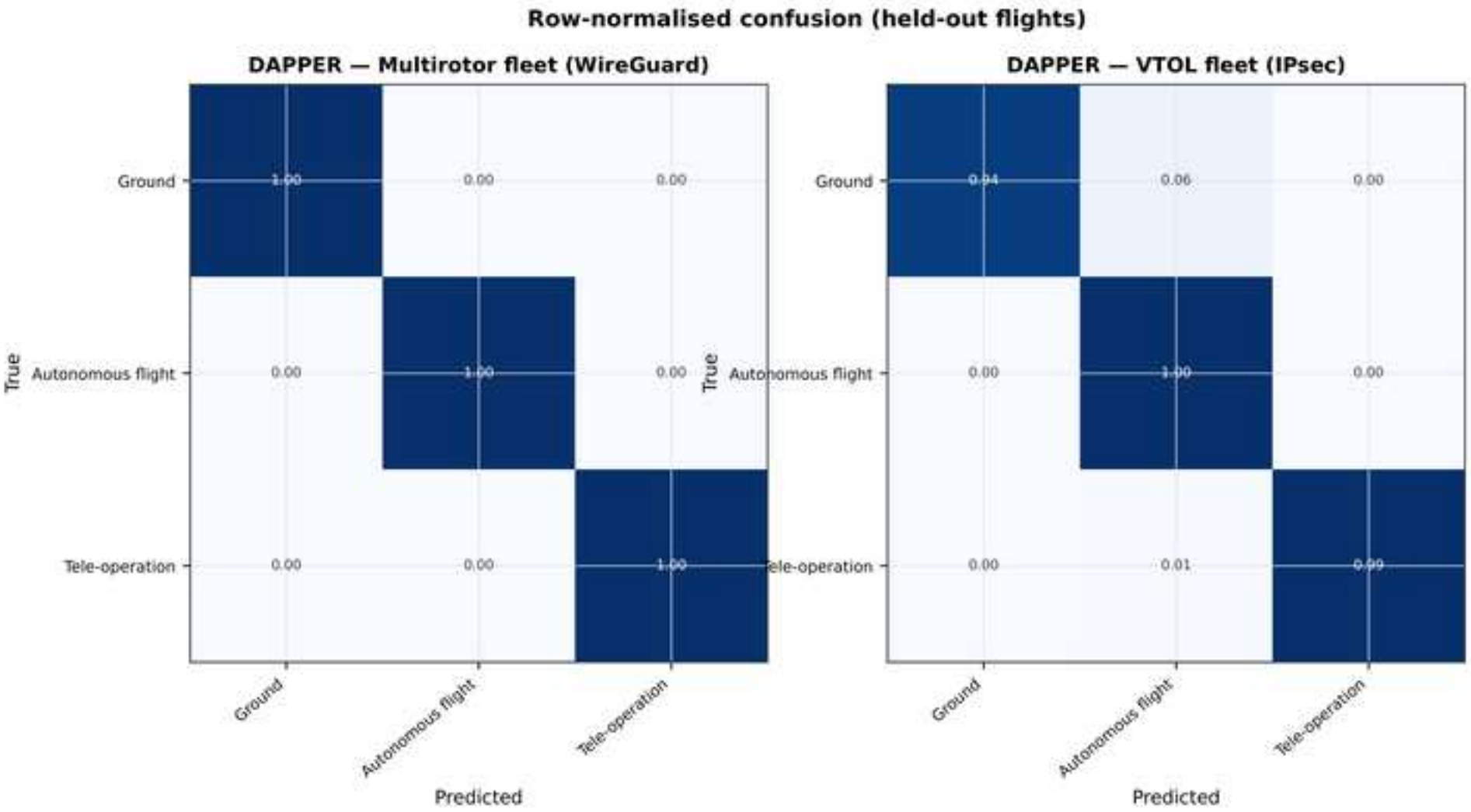
Click here to access/download;Figure;Fig4.tif



Click here to access/download;Figure;Fig5.tif



Click here to access/download;Figure;Fig6.tif



Click here to access/download;Figure;Fig7.tif

